Blockchain-Based Patch Management System

Nithya C
Computer Science And Business Systems
Knowledge Institute Of Technology
(An Autonomous Institution)
Salem, India
cncsbs@kiot.ac.in

Raveena K
Computer Science And Business Systems
Knowledge Institute Of Technology
(An Autonomous Institution)
Salem, India
2k22csbs39@kiot.ac.in

Barathi K
Computer Science And Business Systems
Knowledge Institute Of Technology
(An Autonomous Institution)
Salem, India
2k22csbs07@kiot.ac.in

Madhu Varsha P
Computer Science And Business Systems
Knowledge Institute Of Technology
(An Autonomous Institution)
Salem, India
2k22csbs26@kiot.ac.in

Gayathri R
Computer Science And Business Systems
Knowledge Institute Of Technology
(An Autonomous Institution)
Salem, India
2k22csbs14@kiot.ac.in

***Abstract*—** A Blockchain-Based Patch Management System (BPMS) is proposed to address the security issues of traditional centralized patch management systems, such as tampering, spoofing, and single points of failure. The system uses blockchain technology to ensure transparent and immutable tracking of patch data, while the Interplanetary File System (IPFS) enables decentralized storage of patch files. Smart contracts automate update verification and enforce role-based access control. Before installation, devices verify patch integrity using SHA-256 hashing and record the installation status on the blockchain. This approach enhances trust, traceability, and auditability, making it suitable for critical environments like defense systems, enterprise networks, and IoT infrastructures.

Keywords— Blockchain, Patch Management, Smart Contracts, IPFS, Cybersecurity, Decentralized Systems, Software Updates

I. INTRODUCTION

In today's world where everything is connected through technology, software systems are important in almost every area, like healthcare, military, banking, schools, and manufacturing. As we rely more on software systems, the chance of facing cyber threats and security weaknesses becomes much higher. Attackers consistently take advantage of software vulnerabilities to gain unauthorized entry, interfere with services, or steal confidential information.

To reduce these risks, software companies often provide patches and updates that correct mistakes, fix security issues, and make the system work better. Patch management is an important process that helps keep systems safe, current, and dependable. The success of patch management relies on more than just having the patches available; it also depends on keeping the process of distributing the patches secure and reliable.

Traditional patch management systems are primarily centralized, with a central server responsible for distributing updates to client systems. Although this method is commonly used, it has some built-in issues. A centralized system has one main point where it can fail. If that central server is attacked or broken, the whole system is at risk. Attackers can take advantage of this weakness to add harmful updates, which can cause big security problems. Centralized systems lack transparency, making it difficult for users to verify the authenticity and origin of patches.

Another big problem with old systems is that there's no good way to make sure patches aren't changed when they're being sent from one place to another. Even if a patch is legitimately released, it can be modified by attackers before it reaches the end device. Another thing is that there's no standard method to show that patches have been properly installed, which is important for checking and following rules in big company setups.

To address these challenges, emerging technologies such as blockchain offer a promising solution. Blockchain is a decentralized and distributed ledger technology that ensures immutability, transparency, and trust without relying on a central authority. By leveraging blockchain, it is possible to create a secure and tamper-proof system for managing software updates.

In this paper, we propose a Blockchain-Based Patch Management System (BPMS) that enhances the security and reliability of patch distribution. The proposed system utilizes blockchain to store patch metadata, cryptographic hashes, and verification records, ensuring that all transactions are immutable and transparent. Smart contracts are used to automate the validation process and enforce access control policies.

To handle large patch files efficiently, the system integrates the Interplanetary File System (IPFS), a decentralized storage solution that allows secure and distributed file sharing. The actual patch files are stored in IPFS, while only their cryptographic hashes are stored on the blockchain. This hybrid approach ensures scalability and reduces storage costs.

A. Problem Statement

1) Centralized Architecture Limitations:

Most traditional patch management systems use a central server or storage place to send out updates. This makes managing things easier, but it also creates a big security problem called a single point of failure. If the main server is hacked, attackers can change or send harmful updates to all the devices connected to it.

In big company setups, if an update server gets hacked, it can cause many systems to become infected, leading to huge money and business losses.

2) Lack of Patch Authenticity Verification:

In many existing systems, there is no robust mechanism to verify whether a patch originates from a trusted publisher. Attackers can impersonate legitimate vendors and distribute fake updates, leading to serious security breaches.

2.1 Without strong authentication:

- Devices cannot confirm the source of updates
- Users blindly trust the system
- Malicious patches can be installed

3) Risk of Tampering During Transmission:

Even if a patch is legitimate at the source, it can be altered during transmission through network attacks such as:

- Man-in-the-Middle (MITM) attacks
- Packet injection attacks

4) Absence of Verifiable Installation Proof:

Companies usually need to show that updates have been properly installed on every device. However, current systems do not have a dependable and secure way to keep track of:

- Which device installed the patch
- When the installation occurred
- Whether the installation was successful

Logs stored in centralized databases can be modified or deleted, making them unreliable for auditing and compliance purposes.

B. Proposed System:

The proposed system is a Blockchain-Based Patch Management System (BPMS) that ensures secure, transparent, and tamper-proof patch distribution. It integrates blockchain technology with decentralized storage to eliminate the limitations of traditional systems.

In this system, patch publishers upload software updates, which are stored in decentralized storage (IPFS). A cryptographic hash of each patch is generated and stored on the blockchain along with metadata such as version details and publisher identity.

Client devices can check for updates, download patches, and verify their authenticity by comparing the file hash with the blockchain record. Only verified patches are installed, ensuring system security.

Publisher - Uploads and registers patches

- The client's device checks for updates and installs them.
- The admin watches over the system and takes care of managing the users.
- An auditor checks to make sure everything is correct and reviews the records of checks and activities.

II. METHODOLOGY

The proposed Blockchain-Based Patch Management System (BPMS) uses an organized, tech-based approach that combines user-facing tools, server-side functions, blockchain smart contracts, and distributed storage to provide secure and trustworthy patch management.

The system process is broken down into several steps, and each step uses particular tools and methods to make sure it's secure, clear, and works well.

A. Phase 1: System Initialization

This step sets up the system environment and defines the user roles using identity based on blockchain.

1) Step 1: Wallet-Based Authentication

Users (Admin, Publisher, Device) connect using wallets through:

- wagmi
- RainbowKit
- Wallet address acts as unique identity

Internally:

- ethers.js connects frontend to smart contract
- Role is verified using Solidity mappings

2) Step 2: Role Assignment (RBAC)

The system uses smart contracts to enforce Role-Based Access Control (RBAC).

Implementation (Solidity):

- mapping(address => bool) public publishers;
- mapping(address => bool) public devices;

Logic:

Admin → Full access

Publisher → Upload access

Device → Install/report access

3) Step 3: Device Registration & Publisher Approval

- Admin registers devices via frontend (Next.js UI)
- Data stored in:
  - Blockchain (for trust)
  - MongoDB (for fast queries)
- Publishers are approved through smart contract transactions

Technologies used:

- Frontend - Next.js + React
- Backend - Node.js + Express
- Blockchain – Solidity

B. Phase 2: Patch Upload & Publishing

This phase ensures secure patch storage and registration.

1) Step 1: Patch Upload via Frontend

Publisher uploads file using:

- Next.js UI
- Tailwind-styled forms

2) Step 2: Storage in IPFS

- File is uploaded to IPFS via Pinata

Output:

CID (Content Identifier) generated

3) Step 3: SHA-256 Hash Generation

Using Node.js crypto module:

crypto.createHash('sha256').update(file).digest('hex')

Purpose:

- Generate unique fingerprint of file

4) Step 4: Smart Contract Registration

Using ethers.js, system stores:

- IPFS CID
- SHA-256 hash
- Publisher address
- Timestamp

Blockchain Network:

- Base Sepolia

5) Step 5: Backend Logging

- Patch metadata stored in MongoDB using Mongoose
- Ensures fast access and filtering

C. Phase 3: Patch Distribution & Device Execution

This is the core functional phase where real verification happens.

1) Step 1: Device Agent Execution

Device simulated using:

- Node.js script
- File system access

2) Step 2: Fetch Patch Metadata

Device calls backend API (Express.js)

Backend retrieves:

- Patch info from MongoDB
- Hash from blockchain

3) Step 3: Patch Selection Logic

Algorithm:

IF (patch.version > current.version)
AND (platform matches)- proceed

Ensures:

Only relevant updates are installed

4) Step 4: Download from IPFS

- File fetched using CID

5) Step 5: Integrity Verification (CRITICAL)

Using SHA-256:

```
computed_hash = SHA256(downloaded_file)
IF computed_hash == blockchain_hash:
   proceed
ELSE:
   abort
```

Prevents:

- Tampering
- Fake updates

6) Step 6: Patch Installation

Device updates system (simulated)

Example:

- Version number updated
- File replaced

7) Step 7: Reporting to Blockchain

Device sends transaction:

```
reportInstallation(patchId, status)
```

Using:

- ethers.js

Stored in blockchain:

- Device address
- Patch ID
- Status
- Timestamp

8) Step 8: Backend Logging

Logs stored in MongoDB:

- Device ID
- Status
- Time

D. Phase 4: Monitoring and Analytics

1) Step 1: Admin Dashboard

Built using:

- Next.js
- React

Displays:

- Total patches
- Devices updated
- Success rate

2) Step 2: Publisher Dashboard

Shows:

- Patch adoption
- Installation count

3) Step 3: Data Visualization

- Backend APIs provide data
- Frontend renders charts

E. Security Mechanisms in Methodology

1) SHA-256 Hashing

- Ensures file integrity
- Used in verification

2) Blockchain Immutability

- Prevents data modification
- Ensures trust

3) RBAC (Role-Based Access Control)

- Prevents unauthorized actions

4) API Security

- CORS
- Helmet
- dotenv

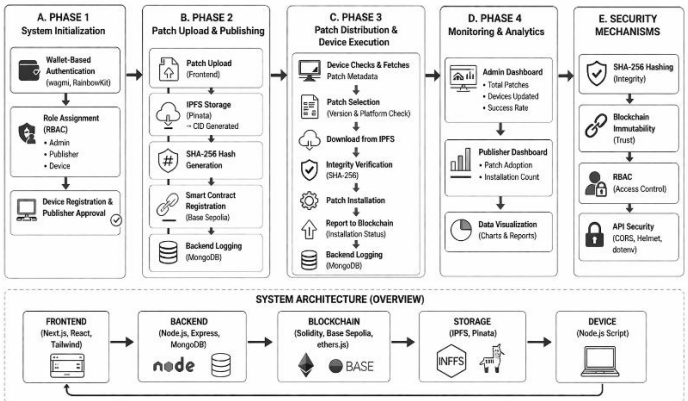

Fig. 1. Overview of Blockchain-Based Patch Management System

III SECURITY MODEL

The proposed Blockchain-Based Patch Management System (BPMS) uses a security model with several layers to make sure software patches are safely shared, sent, checked, and installed. The system uses blockchain technology, cryptographic hashing, and controlled access methods to remove weaknesses found in regular patch management systems. The system uses decentralized trust and solid verification methods to make sure that every part of the patch process is real, reliable, and open to view.

A. Identity Management

The system uses blockchain wallet addresses as unique identities for everyone involved, like the Admin, Publisher, and Device. Every action taken in the system is connected to a particular wallet address, which helps track and take responsibility for each action. Wallet-based authentication uses cryptographic keys, which makes it more secure than regular login methods.

B. Role-Based Access Control (RBAC)

Access to the system's features is limited according to the roles set up in the smart contract. Only certain approved groups can carry out particular tasks.

- Admin: Registers devices and approves publishers
- Publisher: Uploads and publishes patches
- Device: Installs updates and reports status

This stops people who aren't supposed to be there from doing important tasks like putting out updates or installing them.

C. Data Integrity using SHA-256

To stop people from changing patch files, the system uses a strong code called SHA-256 to create a unique fingerprint for each file.

- A hash is created when the patch is uploaded.
- The hash is stored on the blockchain
- The device recalculates the hash once the file has been downloaded.

Both hashes are compared before installation

If the hashes are the same, the file is seen as real; if they are different, the file is not accepted.

D. Blockchain Immutability

All important information like details about software updates and records of when they were installed is kept on

the blockchain. Once the data is stored, it cannot be changed or removed.

- Ensures tamper-proof records
- Provides permanent audit trail
- Increases trust among participants

E. Secure Storage using IPFS

Patch files are kept in IPFS, which is a type of storage system that works without relying on a single central server.

- Files are identified using CID (Content Identifier)
- Any modification changes the CID
- Prevents unauthorized file manipulation

This makes sure the downloaded file is exactly like the uploaded file.

F. API and Backend Security

The backend system has extra security features to keep communication safe.

- CORS: Controls cross-origin requests
- Helmet: Protects against common web vulnerabilities
- dotenv: Secures sensitive configuration data

G. Summary of Security Features

The BPMS security model provides:

- Strong identity verification using wallets
- Controlled access through RBAC
- Tamper detection using SHA-256
- Immutable records using blockchain
- Decentralized storage using IPFS

IV SYSTEM STUDY AND DESIGN

The system study and design phase involves looking closely at the needs of the Blockchain-Based Patch Management System (BPMS) and creating a strong structure that ensures security, scalability, and good performance. The study looks at current patch management systems, finds out what's not working with them, and creates a new solution that fixes those problems by using modern technologies like blockchain and decentralized storage.

A. System Study

Looking at current systems shows that most patch management tools are designed to work from a central location. These systems rely on one server to send out updates, which makes them easy targets for problems and security threats. They also don't have clear transparency and don't offer trustworthy ways to check if a patch is real or keep a record of when it was installed.

Based on this analysis, these important requirements were found:

- A system that is spread out so there is no single point where everything depends on one part.
- Secure authentication and access control
- Mechanism to verify patch integrity
- Transparent and tamper-proof logging
- Scalable architecture for large number of devices

The new BPMS is made to meet these needs by using blockchain to build trust, IPFS for storing data, and backend systems to ensure good performance.

B. System Design

The system design explains how various parts work together to perform the needed tasks. The BPMS is built with a modular and layered structure, and each part has a particular job to do.

- The design includes three major actors:
- The administrator is responsible for handling the system by adding devices and giving permission to publishers.
- Publisher uploads and publishes patches
- Device installs patches and reports status

The whole process is set up to make sure that communication and checking are safe and reliable at each step. When a publisher uploads a patch, the file is saved in IPFS and its unique identifier is written onto the blockchain. Devices get the patch information, download the file, check if it's genuine, and install it only if it's real. Once the installation is done, the status is stored on the blockchain, which makes it clear and easy to check later.

C. Functional Design

The functional design shows how the system works and what it does.

- Device registration and publisher approval
- Patch upload and publishing
- Patch verification and installation
- Status reporting and monitoring

Each function is built using a mix of frontend, backend, and blockchain parts to make sure it works well and stays secure.

D. Non-Functional Design

The system also takes into account non-functional requirements such as:

- Security: Ensured using blockchain and cryptographic hashing
- Scalability is made possible by using decentralized storage and a modular architecture.
- Performance: Improved using backend APIs and database
- Reliability is ensured by using distributed systems like IPFS and blockchain.

E. Design Advantages

The proposed design provides several benefits:

- Eliminates central dependency
- Ensures tamper-proof operations
- Supports large-scale environments
- Provides transparent and verifiable logs

V CONCLUSION

The new Blockchain-Based Patch Management System (BPMS) offers a safe and dependable way to handle updates in modern systems by fixing the main problems with old, centralized methods. Current solutions usually have problems like not being clear, being easy to change, and not having proof that updates have been properly applied. The system tackles these issues by using blockchain, decentralized storage, and cryptographic checks to ensure security and reliability.

In this system, information about software patches and verification results is safely kept on the blockchain, making it unchangeable and trustworthy. Using SHA-256 hashing ensures that patch files remain unchanged, so devices can check if they are real before installing them. IPFS is used to store patch files in a decentralized way, which means it doesn't rely on a single central server and makes the system more scalable. Role-Based Access Control makes sure that only approved publishers and devices can use the system, which helps keep the system safer overall.

The method used in BPMS makes sure that the entire workflow is secure from start to finish, with patches being released, checked, put into place, and kept track of in a clear and unchangeable way. The system also keeps a record of all actions taken, allowing admins to check what patches were deployed and make sure everything follows the rules.

The suggested BPMS shows how using blockchain, IPFS, and current web tools can build a safe, decentralized, and effective system for managing software updates. This method helps build more trust and makes the system dependable, which is why it works well in important areas like business systems, smart device networks, and security setups were keeping things safe is essential.

VI FUTURE WORK

Although the Blockchain-Based Patch Management System (BPMS) offers a secure and decentralized way to handle patch management, there are several areas where the system could be improved to make it more efficient, smart, and suitable for real-world use. Future updates could include automation, better scaling, and more detailed analysis to make the system stronger and better able to handle new needs as they come up.

One key improvement is the use of Artificial Intelligence and Machine Learning methods. These tools can help automatically check for weaknesses, decide which fixes are most important, and guess possible security problems before they happen. Using AI helps the system cut down on manual work and makes it easier to make better decisions when applying software updates.

Another thing that could be improved is using automatic systems to plan and send out software updates. Right now, installing a patch needs the user to take action, but in upcoming versions, the system will be able to update patches automatically in the background according to set rules. This will help apply updates on time, especially in big setups.

The system can also be expanded to work with multiple blockchains or across different chains. Using more than one blockchain network instead of just one can make the system work better, lower the cost of each transaction, and make it easier to adapt and change as needed. This method can also help prevent problems with the network getting too busy.

Connecting with cloud services like AWS or Azure could be another useful improvement. This would allow the system to be used widely across different setups and help make it more reliable and faster. Cloud integration can also help with backup, balancing the load, and recovering from disasters.

Future work could also involve creating a dashboard that provides real-time analytics and monitoring. Advanced visualization tools can help you understand how quickly patches are being applied, what problems are happening with updates, and how well the system is working. This will allow admins to make better choices and spot possible problems faster.

Another improvement is making the device more automatic and smarter. Devices can be made to automatically check for updates, verify their safety, and apply fixes without needing someone to do it manually. You can also create lighter agents that work well with IoT devices that have fewer resources.

The system can also include ways to let users know about new changes, installation progress, or problems, like sending emails or alerts on a phone. This improves user awareness and responsiveness.

Finally, security can be made even stronger by using advanced methods like zero-knowledge proofs or secure multi-party computation, which help protect privacy and keep data safe within the system.

VII REFERENCES

[1] M. Conti, S. Kumar, C. Lal, and S. Ruj, “A Survey on Security and Privacy Issues of Blockchain Technology,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 341–368, 2021.

[2] Y. Zhang, J. Chen, and X. Wang, “Blockchain-Based Secure Software Update Framework for IoT Devices,” *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4379–4390, 2021.

[3] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” *Decentralized Business Review*, 2020 (reprint and analysis version).

[4] A. Dorri, S. S. Kanhere, and R. Jurdak, “Blockchain for IoT Security and Privacy: The Case Study of a Smart Home,” *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 5513–5524, 2020.

[5] H. Shafagh, L. Burkhalter, A. Hithnawi, and S. Duquennoy, “Towards Blockchain-Based Auditable Storage and Sharing of IoT Data,” *IEEE Cloud Computing*, vol. 7, no. 2, pp. 48–57, 2020.

[6] K. Christidis and M. Devetsikiotis, “Blockchains and Smart Contracts for the Internet of Things,” *IEEE Access*, vol. 9, pp. 117911–117927, 2021.

[7] P. Zhang, M. A. Walker, and J. White, “Security and Trust Mechanisms in Decentralized Systems,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 3, pp. 1452–1465, 2022.

[8] R. Roman, J. Zhou, and J. Lopez, “On the Features and Challenges of Security and Privacy in Distributed Internet of Things,” *Computer Networks*, vol. 215, pp. 109–124, 2022.

[9] S. Li, L. Da Xu, and S. Zhao, “The Internet of Things: A Survey,” *IEEE Communications Surveys & Tutorials*, vol. 24, no. 1, pp. 102–130, 2023.

[10] A. Singh and K. Chatterjee, “A Survey on Blockchain Security Issues and Challenges,” *Journal of Network and Computer Applications*, vol. 199, 2023.

[11] J. Chen, X. Wu, and Y. Li, “Secure and Efficient Software Update Mechanism Using Blockchain,” *IEEE Access*, vol. 11, pp. 55678–55690, 2023.

[12] M. Patel and R. Gupta, “Decentralized File Storage Using IPFS and Blockchain Integration,” *International Journal of Information Security*, vol. 22, no. 4, pp. 567–580, 2024.